

Week 2 Exercises

Nischal Pant

May 14, 2022

Please complete all exercises below. You may use stringr, lubridate, or the forcats library.

```
library(stringr)
library(lubridate)
```

```
## Warning: package 'lubridate' was built under R version 4.2.3
##
## Attaching package: 'lubridate'
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
library(forcats)
```

```
## Warning: package 'forcats' was built under R version 4.2.3
```

Exercise 1

Read the sales_pipe.txt file into an R data frame as sales.

```
sales <- read.delim("C:/Users/Nischal/Downloads/sales_pipe.txt",
  header = TRUE,
  check.names = FALSE,
  stringsAsFactors=FALSE,
  sep = "|")
```

Exercise 2

You can extract a vector of columns names from a data frame using the colnames() function. Notice the first column has some odd characters. Change the column name for the FIRST column in the sales data frame to Row.ID.

Note: You will need to assign the first element of colnames to a single character.

```
#first find the names of the column headers
colnames(sales)
```

```
## [1] "\xef..Row.ID" "Order.ID" "Order.Date" "Ship.Date"
## [5] "Ship.Mode" "Customer.ID" "Customer.Name" "Segment"
## [9] "Country" "City" "State" "Postal.Code"
## [13] "Region" "Product.ID" "Category" "Sub.Category"
## [17] "Product.Name" "Sales" "Quantity" "Discount"
## [21] "Profit"
```

```
#it shows that one of them has special characters, now lets change that to "Row.ID"
colnames(sales)[colnames(sales) == "\xef..Row.ID"] <- "Row.ID"
```

```
#check the names again to check if it has now been changed. It has.
colnames(sales)
```

```
## [1] "Row.ID"      "Order.ID"      "Order.Date"     "Ship.Date"
## [5] "Ship.Mode"     "Customer.ID"    "Customer.Name"  "Segment"
## [9] "Country"       "City"           "State"          "Postal.Code"
## [13] "Region"        "Product.ID"     "Category"       "Sub.Category"
## [17] "Product.Name"  "Sales"          "Quantity"       "Discount"
## [21] "Profit"
```

Exercise 3

Convert both Ship.Date and Order.Date to date vectors within the sales data frame. What is the number of days between the most recent order and the oldest order? How many years is that? How many weeks?

Note: Use lubridate

```
#first lets use lubridate's mdy function to change all the date format and plug them back into the sales
sales$Ship.Date <- mdy(sales$Ship.Date)
sales$Order.Date <- mdy(sales$Order.Date)
```

```
#summary to give details about the Order.date variable of the sales data set
summary(sales$Order.Date)
```

```
##           Min.          1st Qu.          Median            Mean          3rd Qu.           Max.
## "2014-01-03" "2015-06-11" "2016-07-17" "2016-05-09" "2017-05-28" "2017-12-30"
```

```
#now lets extract that variable and its cells and assign them to variable dates
dates <- ymd(sales$Order.Date)
```

```
#using the intervals function lets assign them to date_intervals to calculate how many days are between
date_interval <- interval(min(dates), max(dates))
```

```
#because the data is in date format, change them into days.
num_days <- as.integer(date_interval / ddays(1))
```

```
#number of days
num_days
```

```
## [1] 1457
```

```
#calculation for number of years
num_days/365
```

```
## [1] 3.991781
```

```
#calculation for number of weeks
num_days/7
```

```
## [1] 208.1429
```

Exercise 4

What is the average number of days it takes to ship an order?

```
#for this we need number of days between the order.date variable and ship.date variable. Lets assign th
subtracted_values <- sales$Ship.Date - sales$Order.Date
mean(subtracted_values)

## Time difference of 3.908482 days
```

Exercise 5

How many customers have the first name Bill? You will need to split the customer name into first and last name segments and then use a regular expression to match the first name bill. Use the length() function to determine the number of customers with the first name Bill in the sales data.

```
#lets first create a matrix with the str_split function to split the name string into two cells separat
name <- str_split_fixed(sales$Customer.Name, " ", n = 2)

#now using which, we can find how many indices exist where the first name is Bill and the length functi
length(which(name[, 1] == "Bill"))

## [1] 37
```

Exercise 6

How many mentions of the word 'table' are there in the Product.Name column? **Note you can do this in one line of code**

```
#we can use the str_count to find how many times table shows up in the sales data set
sum(str_count(sales$Product.Name, fixed('table'))))

## [1] 240
```

Exercise 7

Create a table of counts for each state in the sales data. The counts table should be ordered alphabetically from A to Z.

```
#the code I originally wrote was concise and only 2 steps but failed to sort them alphabetically. They

# first lets create a table with the states in sales data set
states <- table(sales$State)

# Sort the counts table alphabetically
sorted_state <- sort(states)

# now we create a data frame with state and count columns
states_final <- data.frame(State = names(sorted_state), Count = sorted_state)

# now we organize them again by order
states_final <- states_final[order(states_final$State), ]

states_final

##           State      Count.Var1 Count.Freq
```

## 21	Alabama	Alabama	28
## 40	Arizona	Arizona	119
## 15	Arkansas	Arkansas	22
## 49	California	California	993
## 38	Colorado	Colorado	90
## 28	Connecticut	Connecticut	50
## 27	Delaware	Delaware	47
## 1	District of Columbia	District of Columbia	1
## 42	Florida	Florida	186
## 35	Georgia	Georgia	79
## 7	Idaho	Idaho	9
## 45	Illinois	Illinois	286
## 34	Indiana	Indiana	74
## 11	Iowa	Iowa	11
## 13	Kansas	Kansas	16
## 32	Kentucky	Kentucky	64
## 14	Louisiana	Louisiana	18
## 4	Maine	Maine	4
## 31	Maryland	Maryland	63
## 33	Massachusetts	Massachusetts	71
## 41	Michigan	Michigan	142
## 26	Minnesota	Minnesota	41
## 19	Mississippi	Mississippi	27
## 23	Missouri	Missouri	37
## 3	Montana	Montana	2
## 18	Nebraska	Nebraska	26
## 16	Nevada	Nevada	24
## 8	New Hampshire	New Hampshire	9
## 30	New Jersey	New Jersey	58
## 12	New Mexico	New Mexico	11
## 48	New York	New York	555
## 39	North Carolina	North Carolina	117
## 6	North Dakota	North Dakota	7
## 43	Ohio	Ohio	211
## 24	Oklahoma	Oklahoma	38
## 29	Oregon	Oregon	56
## 46	Pennsylvania	Pennsylvania	312
## 17	Rhode Island	Rhode Island	25
## 22	South Carolina	South Carolina	28
## 9	South Dakota	South Dakota	9
## 37	Tennessee	Tennessee	88
## 47	Texas	Texas	460
## 20	Utah	Utah	27
## 10	Vermont	Vermont	10
## 36	Virginia	Virginia	80
## 44	Washington	Washington	254
## 5	West Virginia	West Virginia	4
## 25	Wisconsin	Wisconsin	38
## 2	Wyoming	Wyoming	1

Exercise 8

Create an alphabetically ordered barplot for each sales Category in the State of Texas.

```

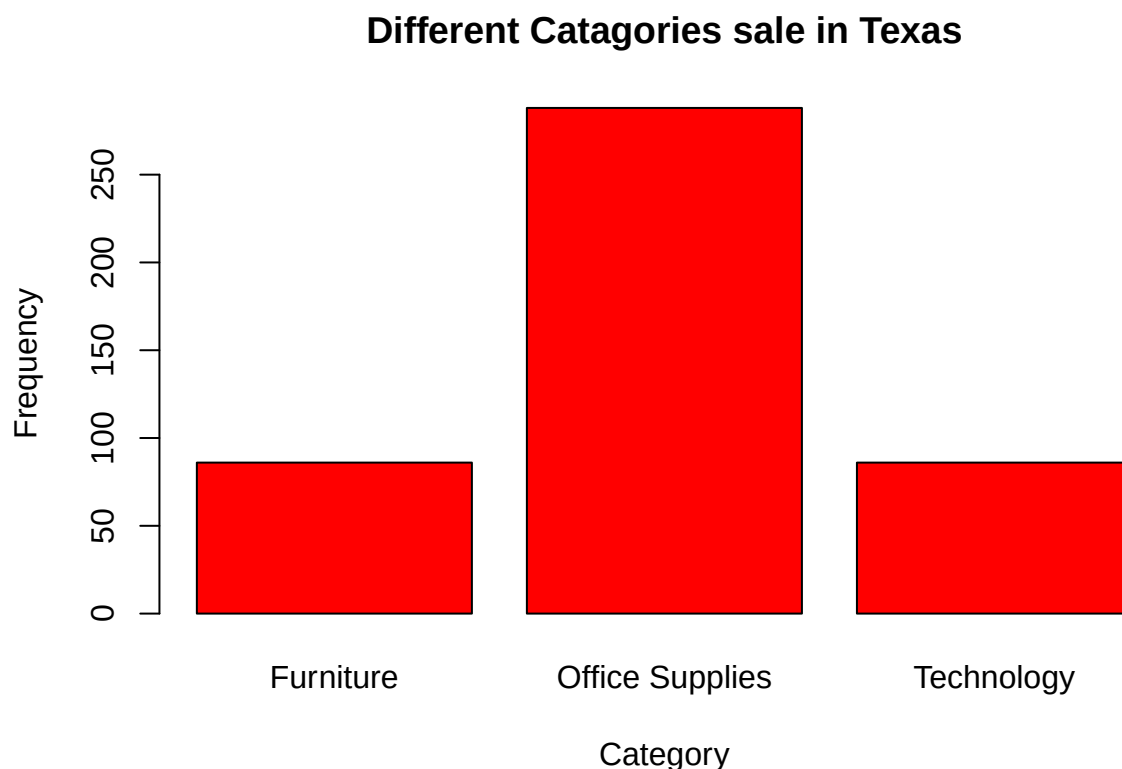
#lets first extract sales of Texas from the sales data set
sales_texas <- sales[sales$State == 'Texas',]

#now lets extract the category variable of that table
category_freq <- table(sales_texas$Category)

#we can now order them alphabetically
category_freq <- category_freq[order(names(category_freq))]

#we can now use the bar plot function to create a bar plot of the data,
barplot(category_freq,
        main = "Different Catagories sale in Texas",
        xlab = "Category",
        ylab = "Frequency",
        col = "RED",
        border = "black")

```



Exercise 9

Find the average profit by region. Note: You will need to use the `aggregate()` function to do this. To understand how the function works type `?aggregate` in the console.

```

#this code aggregates the profit depending on the region and FUN (= function) which is sum in this case

prof <- aggregate(Profit ~ Region, data = sales, FUN = sum, na.action = na.omit)
prof

```

```
##      Region    Profit
## 1 Central 24254.845
## 2     East 42485.498
## 3    South  8311.297
## 4     West 51973.214
```

Exercise 10

Find the average profit by order year. **Note:** You will need to use the `aggregate()` function to do this. To understand how the function works type `?aggregate` in the console.

```
# Your code here
#first lets convert the order.date variable in sale data into year
sales$Order.Date <- year(sales$Order.Date)

#this code aggregates the profit depending on the years and FUN (= function) which is sum in this case.
dateprof <- aggregate(Profit ~ Order.Date, data = sales, FUN = sum, na.action = na.omit)
dateprof
```

```
##   Order.Date    Profit
## 1      2014 31052.73
## 2      2015 21824.21
## 3      2016 38269.30
## 4      2017 35878.62
```